Software Testing and Validation 2025/26
Instituto Superior Técnico
2nd Project Specification

Submission Date: June 5, 2026

1 Introduction

The development of software systems requires a precise and rigorous specification of their functionality and behaviour. Such specifications are essential not only for the development team, but also for the testing team. The project for the Software Testing and Validation course consists of designing test cases based on a provided specification. This specification describes a number of entities within an application.

The primary learning objective of this project is for students to acquire practical experience in test case design by applying the techniques covered in lectures to a set of classes and methods whose behaviour is formally described in this document.

This document is structured as follows. Section 2 describes the entities involved in the system under test, together with implementation details relevant for test design. Section 3 specifies the required tests and the assessment criteria.

2 Problem Description

Consider an urban delivery system that allows users to place orders from restaurants or local shops. An order is represented by the class `Order` and consists of a collection of items. Each item is represented by the class `Item` and has a description, price, weight and is associated with a single shop or a restaurant (Both represented by the `Supplier` class).

Once an order has been created, it may result in a delivery, represented by the class `Delivery`, which models the process of transporting the order to the customer. The delivery is carried out by a courier (represented by the class `Courier`). The main entities of the system are described below.

2.1 The Order entity

The class `Order` represents a customer order composed of multiple items. All items within an order must belong to the same supplier. Each item has an associated quantity, which must be between 1 and 10 units. The total cost of an order is determined by the unit price of each item and the corresponding quantities. The total cost must not exceed 1000 euros. The total number of units in an order (considering all item quantities) must not exceed:

$$20 + \left\lfloor \frac{orderCost}{50} \right\rfloor$$

Each order also has an associated delivery distance, corresponding to the distance between the supplier and the delivery location. This distance is expressed in metres. This distance is assumed to be computed by another system component. The delivery address is provided when the order is created and may be updated subsequently. The delivery distance must not exceed 20 km. The total weight of an order corresponds to the sum of the weights of all item units.

Listing 1 presents the interface of this class. Any method invocation that would place the order in an invalid state must be aborted by throwing an `InvalidOperationException`, and the state of the order must remain unchanged.

Listing 1: Interface of the *Order* class.

```
package supereats.core;

/**
 * This class represents a order.
 **/

public class Order {
  public Order(Client client, String deliveryAddress, int distance) { /* ... */ }

  public void addItem(Item item, int quantity) { /* ... */ }
  public void removeItem(Item item, int quantity) { /* ... */ }
  public void setAddress(String newDeliveryAddress, int distance) { /* ... */ }

  public int quantity(Item item) { /* ... */ }
  public boolean contains(Item item) { /* ... */ }
  // returns the total number of unities of all itens in this order
  public int totalQuantity() { /* ... */ }
  // The cost of an order does not include the delivery cost, it just reflect the cost of
  // the selected itens and corresponding quantities
  public double cost() { /* ... */ }
  // the content of this order
  public List<Pair<Item, Integer>> itens() { /* ... */ }
  public double computeDeliveryCost() { /* ... */ }
}
```

Delivery cost computation The method `computeDeliveryCost()` calculates the delivery cost based on the delivery distance, total order cost, number of units, and total weight. The delivery cost is defined as follows:

- **Distances below 5 km:**
  - If the order cost is less than 75 euros, the delivery cost is:
    - 5 euros if the number of units is greater than 10;
    - 3 euros otherwise.
  - If the order cost is at least 75 euros, the delivery cost is 0 euros.
- **Distances between 5 and 15 km:**
  - If the order cost exceeds 150 euros, the delivery cost is 1 euro.
  - If the order cost is between 75 and 150 euros, the delivery cost is 3 euros.
  - If the order cost is below 75 euros, the delivery cost is:
    - 6 euros if the number of units exceeds 5;
    - 4 euros otherwise.
- **Distances above 15 km and up to 20 km:**
  - If the order cost exceeds 500 euros, the delivery cost is 1 euro.
  - If the order cost is between 300 and 500 euros, the delivery cost is:
    - 3 euros if the order weight is less than 5 kg;
    - 5 euros otherwise.
  - If the order cost is less than 300 euros, the delivery cost is:
    - 7 euros if the weight is less than 3 kg;
    - 8 euros if the weight is at least 3 kg and the number of units is less than 8;
    - 10 euros if the weight is at least 3 kg and the number of units is at least 8.

2.2 The Delivery entity

The class `Delivery` models the delivery process of an order. The delivery is carried out by a courier, assigned during the delivery process, to the address associated with the order. The delivery address may be changed, provided that the delivery has not yet been assigned to a courier. Listing 2 presents the interface of this class.

Listing 2: Interface of the *Delivery* class.

```
package supereats.core;

/**
 * This class represents a deliver.
 **/

enum DeliverMode {
  IN_PREPARATION, READY_TO_DELIVER, IN_TRANSIT, DELIVERED, NOT_DELIVERED, CANCELLED;
}

public class Delivery {
  public Delivery(Order order) { /* ... */ }

  public void changeAddress(String newAddress) { /* ... */ }
  public void assign(Courier courier) { /* ... */ }
  public void setReady() { /* ... */ }
  public void delivered() { /* ... */ }
  public void cancel() { /* ... */ }
  public boolean updateItem(Item item, int quantity) { /* ... */ }

  List<Pair<Item,Integer>> content() { /* ... */ }
  DeliverMode mode() { /* ... */ }
}
```

A delivery may be in one of the following modes:

- `IN_PREPARATION`
- `READY_TO_DELIVER`
- `IN_TRANSIT`
- `DELIVERED`
- `NOT_DELIVERED`
- `CANCELLED`

The current mode of a delivery influences the behaviour of its methods. The following restrictions apply to method invocations in the `Delivery` class:

- Upon creation, a delivery is *in preparation* (mode `IN_PREPARATION`). Once all items are ready for delivery, the method `setReady()` must be invoked, transitioning the delivery to the *ready* mode (`READY_TO_DELIVER`).

  A delivery *in preparation* or *ready* may be cancelled via the method `cancel()`, transitioning to `CANCELLED`.

  In the `IN_PREPARATION` and `READY_TO_DELIVER` modes it is possible to:

  – change the address via `changeAddress()`;
  – update item quantities via `updateItem()`.

  If `updateItem()` is successfully invoked on a *ready* delivery (it returned `true`) with a positive quantity, the delivery returns to *in preparation*, otherwise it remains in the same mode.

- A *ready* delivery may be assigned to a courier via `assign()`, transitioning to *in transit* (`IN_TRANSIT`). This assignment is only valid if the courier has fewer than 5 assigned deliveries.
- An *in transit* delivery is completed successfully by invoking `delivered()`, transitioning to *delivered* (`DELIVERED`).

  Alternatively, if delivery cannot be completed successfully, the method `cancel()` must be invoked. In this case, the delivery transitions to mode `NOT_DELIVERED` if the number of attempts is fewer than 3 attempts; otherwise it transtions to mode `CANCELLED`.
- When a delivery in mode `NOT_DELIVERED` becomes ready for a new attempt, `setReady()` must be invoked, transitioning it back to `READY_TO_DELIVER`.

If a method is invoked outside the allowed context, it must have no effect and must throw the *InvalidInvocationException* exception. The methods `mode()` and `content()` may be invoked at any time and return the mode and content of the invoked delivery.

The method `updateItem()` updates the quantity of an item in the delivery. If its invocation is valid, it returns a boolean indicating whether the operation was successful. If unsuccessful, the delivery content must remain unchanged. The maximum quantity per item (see section 2.1) must always be respected. The specified quantity may be negative, corresponding to item removal. After execution, the number of units must always remain strictly positive, otherwise the operation fails (it is unsuccessful). At most three successful updates are allowed; afterwards, any valid call of this method is always unsuccessful (it returns `false` without modifying the delivery).

3 Project Evaluation

All test cases must be designed using the most most appropriate testing design strategy for each situation. The project requires the specification of test cases for selected classes and methods, as well as the implementation of a subset of them. The test cases to be specified are:

- Class scope test cases for `Order` class (0–3.5 marks).
- Class scope test cases for `Delivery` class (0–6.5 marks).
- Method scope test cases for the `computeDeliveryCost()` method of the `Order` class (0–3.5 marks).
- Method scope test cases for the `updateItem()` method of the `Delivery` class (0–3.5 marks).

Additionally, 8 test cases (4 successful and 4 unsuccessful) must be implemented from the test suite designed to exercise the `Client` class at class scope using the TestNG framework (0–3 marks).

If the *Category Partition* test design strategy is used, only the first 30 combinations need to be presented if more exist, along with the total number of combinations. If the *Combinational Function Testing* test design strategy is used, you only have to show the domain matrices for the first 8 variants if more exist. For each method or class under test, the following must be indicated:

- The name of the test design strategy used;
- Where applicable, the result of each step of the applied strategy using the format presented in lectures;
- The description of the test cases resulting from the chosen testing strategy (typically the result of the final step).

If any of the above points are only partially met, the score will be proportional to the completion.

3.1 Test Case Implementation

For the implementation of test cases, it is not required to provide full implementations of the application classes under test, namely: `Delivery`, `Order`, `Item`, and `Supplier`.

Students are expected to write test cases that are syntactically valid and consistent with the interfaces of these classes. It is not necessary to execute the implemented test cases.

The provided interfaces should be considered sufficient to instantiate the required objects for testing purposes. In particular, Listing 3 presents the interfaces required to use the classes `Item` and `Supplier` when implementing the test cases.

Listing 3: Interfaces of the *Item* and *Supplier* classes.

```
package supereats.core;

/**
 * This class represents an Item.
 **/

public record Item (int weight, int price, Supplier supplier, String description) { }


/**
 * This class represents a Supplier. A supplier has a name.
 **/

public class Supplier {
  public Supplier(String name) { /* ... */ }

  // Returns true if other is a Supplier with the same name as this supplier
  public boolean equals(Object other) { /* ... */ }

  // other methods
  ...
}
```

Students can consider thar the weight of an item is expressed in grams, and the price is specified as an integer value.

Students should assume that all methods behave according to their specification, and no additional implementation details are required.

3.2 Fraud and Plagiarism

Submitting a project implies a **statement of honor** that the submitted work was done by the students listed in the files/documents. Violation of this agreement—i.e., attempting to appropriate work done by others—will result in failing the course for all involved students (including those who enabled it).

3.3 Final Remarks

An oral discussion may be required to assess students' understanding. Students whose exam performance differs significantly from their project grade may be required to attend.

All project-related information is available on the course webpage under *Project*. The submission protocol is also provided there.

Good work!